

# OMNIQUANT: OMNIDIRECTIONALLY CALIBRATED QUANTIZATION FOR LARGE LANGUAGE MODELS

26/05/29

OmniQuant (PTQ)

## [기존 문제]

### 1. Handcraft Quantization Parameter

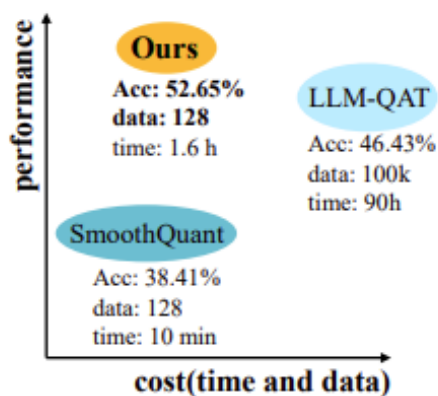
A. Scaling Factor, Zero point 등을 manual하게 계산

### 2. Time intensive gradient optimization

A. AdaRound, BRECQ 등은 gradient optimization을 쓰지만 이는 모든 weight에 대해 계산을 하다 보니 시간이 오래 걸림

Handcraft 방식으로 parameter를 구하는 것은 low bit로 갈수록 성능 저하가 커짐.

하지만 이를 기존 방식처럼 모든 weight에 대한 최적화를 고려하는 것은 또 너무 오래 걸리더라.

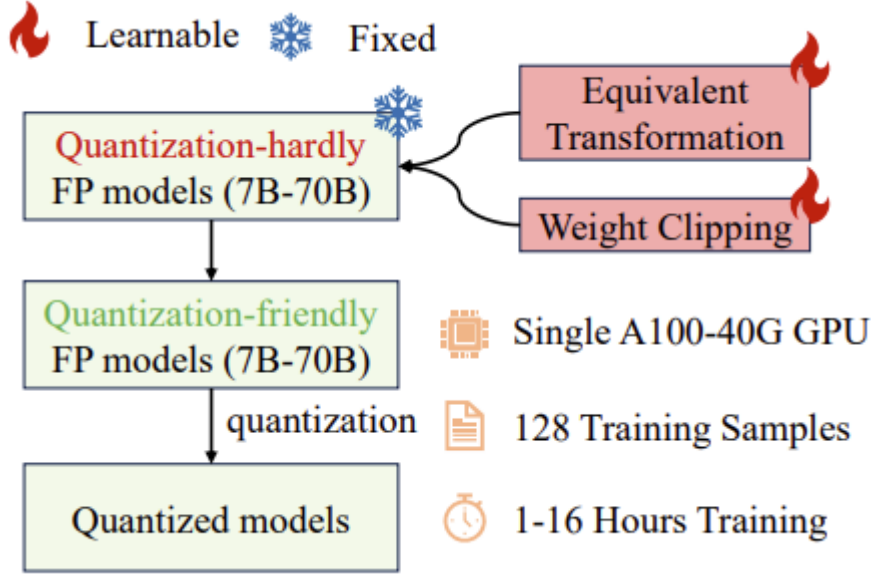


(a) an overview in W4A4

=> Trade-Off

QAT의 성능을 도달하면서 동시에 PTQ처럼 효율적으로 양자화할 수는 없을까?

⇒ **OmniQuant:** 모든 weight에 대한 최적화를 하지 말고,  
Quantization Friendly 하도록 Weight와 Activation을 바꾸는 소수의  
파라미터만 Gradient Optimization으로 학습하자!



## Optimization Goal

quantization parameters can be optimized in a differentiable manner. We formulate the optimization goal as follows:

$$\arg \min_{\Theta_1, \Theta_2} \|\mathcal{F}(\mathbf{W}, \mathbf{X}) - \mathcal{F}(Q_w(\mathbf{W}; \Theta_1, \Theta_2), Q_a(\mathbf{X}, \Theta_2))\|, \quad (1)$$

where  $\mathcal{F}$  represents the mapping function for a transformer block in the LLM,  $\mathbf{W}$  and  $\mathbf{X}$  are full-precision weight and activation,  $Q_w(\cdot)$  and  $Q_a(\cdot)$  represent weight and activation quantizer, respectively,  $\Theta_1$  and  $\Theta_2$  are quantization parameters in learnable weight clipping (LWC) and learnable equivalent transformation (LET), respectively. The Block-wise quantization in Eqn.(1) sequentially quantizes the parameters of one transformer block before moving on to the next.

### 3.2 LEARNABLE WEIGHT CLIPPING

OmniQuant employs a module of learnable weight clipping (LWC) to reduce the difficulty of quantizing the weights in an LLM. Similar to previous methods with learnable clipping threshold (Esser et al., 2019; Liu et al., 2022; Choi et al., 2018), LWC also determines the optimal dynamic range of the weights by optimizing a clipping threshold. However, we find that directly employing prior arts such as PACT (Choi et al., 2018) and LSQ (Esser et al., 2019) in quantization would produce unsatisfactory performance, as demonstrated in Table A14 in the Appendix.

Instead of directly learning a clipping threshold as in previous methods (Esser et al., 2019; Choi et al., 2018), LWC optimizes a clipping strength as formulated by

$$\mathbf{W}_q = \text{clamp}(\lfloor \frac{\mathbf{W}}{h} \rfloor + z, 0, 2^N - 1), \text{ where } h = \frac{\gamma \max(\mathbf{W}) - \beta \min(\mathbf{W})}{2^N - 1}, z = -\lfloor \frac{\beta \min(\mathbf{W})}{h} \rfloor \quad (2)$$

where  $\lfloor \cdot \rfloor$  indicates round operation.  $N$  is the target bit number.  $\mathbf{W}_q$  and  $\mathbf{W}$  denote the quantized and full-precision weights, respectively.  $h$  is the normalization factor for weights and  $z$  is the zero-point value. The clamp operation constrains the value within the range of  $N$ -bit integer, specifically  $[0, 2^N - 1]$ . In Eqn.(2),  $\gamma \in [0, 1]$  and  $\beta \in [0, 1]$  are learnable clipping strengths for the upper and the lower bound of weights, respectively. We instantiate  $\gamma$  and  $\beta$  by the sigmoid function\*. Hence,  $\Theta_1 = \{\gamma, \beta\}$  in Eqn.(1).

Note that LWC degrades into a vanilla MinMax quantization scheme used in existing works (Xiao et al., 2023), Frantar et al. (2022) when  $\gamma = 1$  and  $\beta = 1$ . By inheriting the benefits of MinMax quantization, LWC only needs to adjust the clipping strengths to determine an optimal clipping threshold, which would reduce the optimization difficulty. Clipped by an optimal threshold, the original weights would be easy to quantize. As indicated by the experiments in Table 1, our proposed learnable weight clipping method significantly outperforms previous weight-only quantization techniques (Frantar et al., 2022; Lin et al., 2023)).

## LWC

**(r, B) 를 학습**

**Linear layer.** The linear layer takes an input token sequence  $\mathbf{X} \in \mathbb{R}^{T \times C_{in}}$  where  $T$  is the token length and is the multiplication of the weight matrix  $\mathbf{W} \in \mathbb{R}^{C_{in} \times C_{out}}$  and bias vector  $\mathbf{B} \in \mathbb{R}^{1 \times C_{out}}$ . A mathematically equivalent linear layer is expressed as:

$$\mathbf{Y} = \mathbf{X}\mathbf{W} + \mathbf{B} = \underbrace{[(\mathbf{X} - \delta) \oslash \mathbf{s}]}_{\tilde{\mathbf{X}}} \cdot \underbrace{[\mathbf{s} \odot \mathbf{W}]}_{\tilde{\mathbf{W}}} + \underbrace{[\mathbf{B} + \delta \mathbf{W}]}_{\tilde{\mathbf{B}}} \quad (3)$$

where  $\mathbf{Y}$  represents the output,  $\mathbf{s} \in \mathbb{R}^{1 \times C_{in}}$  and  $\delta \in \mathbb{R}^{1 \times C_{in}}$  are channel-wise scaling and shifting parameters, respectively,  $\tilde{\mathbf{X}}$ ,  $\tilde{\mathbf{W}}$  and  $\tilde{\mathbf{B}}$  are equivalent activation, weight and bias, respectively, ' $\oslash$ ' and ' $\odot$ ' are elementwise division and multiplication. By Eqn.(3), the activations are transformed to be quantization-friendly at a cost of increased quantization difficulty in weights. In this sense, LWC in Sec. 3.2 can improve the performance of weight-activation quantization achieved by LET because it renders weights quantization-friendly. Finally, we perform quantization on transformed activations and weights, as given by

$$\mathbf{Y} = Q_a(\tilde{\mathbf{X}})Q_w(\tilde{\mathbf{W}}) + \tilde{\mathbf{B}}, \quad (4)$$

where  $Q_a$  is the vanilla MinMax quantizer and  $Q_w$  is the MinMax quantizer with learnable weight clipping (i.e. our LWC).

Note that the scaling and shifting parameters in  $\tilde{\mathbf{X}}$  can be absorbed into the previous normalization or linear layer and the scaling factors in  $\tilde{\mathbf{W}}$  can be fused into the original linear weight  $\mathbf{W}$ . Therefore, the equivalent transformation in Eqn.(3) can effectively reduce quantization errors without introducing additional parameters or costs. We employ this equivalent transformation in all linear layers of the LLM except for the second linear layer of FFN as shown in Figure 3. This may be because the high sparsity of features after the non-linear layer (Liu et al., 2023c) leads to unstable gradients when applying learnable equivalent transformations.

**Attention operation.** Beyond the linear layer, the attention operation also accounts for a significant proportion of the computation. Additionally, the auto-regressive pattern of LLM necessitates storing the key-value(KV) cache for each token, which results in substantial memory demands for long sequences. Therefore, we also quantize  $\mathbf{Q}/\mathbf{K}/\mathbf{V}$  matrixes into low-bit in the weight-activation quantization setting. Specifically, the learnable equivalent transform of the self-attention affinity matrix can be written as:

$$\mathbf{P} = \text{Softmax}(\mathbf{Q}\mathbf{K}^T) = \text{Softmax}(\underbrace{(\mathbf{Q} \oslash \mathbf{s}_a)}_{\tilde{\mathbf{Q}}} \underbrace{(\mathbf{s}_a \odot \mathbf{K}^T)}_{\tilde{\mathbf{K}}^T}). \quad (5)$$

where  $\mathbf{s}_a \in \mathbb{R}^{1 \times C_{out}}$  is the scaling factor in the affinity matrix. Similar to Eqn.(4), the quantized affinity matrix calculation is expressed as  $\mathbf{P} = \text{Softmax}(Q_a(\tilde{\mathbf{Q}})Q_a(\tilde{\mathbf{K}}^T))$ . Here we also use MinMax quantization scheme as  $Q_a$  to quantize  $\tilde{\mathbf{Q}}/\tilde{\mathbf{K}}$  matrixes. From Eqn.(4) and Eqn.(5) we know that  $\Theta_2 = \{\delta, \mathbf{s}, \mathbf{s}_a\}$  in Eqn.(1).

The channel-wise scaling factors in  $\tilde{\mathbf{Q}}$  and  $\tilde{\mathbf{K}}$ , as seen in Eq.(5), can be absorbed into linear weights of the query and key projection, respectively. It is worth mentioning that the explicit transformation of  $\mathbf{V}$  is omitted as its distribution has already been channel-wise altered by the inverse transformation associated with the output projection linear layer.

## LET

$(\delta, \mathbf{s}, \mathbf{s}_a)$ 를 학습

### 외부 차원( $C_o$ ) 스케일링 (가능)

가중치의 각 출력 채널( $j$ )마다 서로 다른 스케일값  $s_{W,j}$ 를 가진다고 가정해봅시다.

$$y_{tj} = \sum_{k=1}^{C_i} (x_{tk} \cdot (w_{kj} \cdot s_{W,j}))$$

여기서  $s_{W,j}$ 는 시그마( $\sum$ ) 연산 동안  $k$ 값에 상관없이 상수입니다. 따라서 괄호 밖으로 쏙 빼집니다.

$$y_{tj} = s_{W,j} \cdot \left( \sum_{k=1}^{C_i} x_{tk} \cdot w_{kj} \right)$$

**결론:** 하드웨어는 괄호 안의 정수(INT8) 연산을 최대 속도로 다 끝낸 뒤, 마지막에 스케일( $s_{W,j}$ )만 딱 한 번 곱해주면 됩니다.

### 내부 차원( $C_i$ ) 스케일링 (불가능)

반면, 액티베이션의 입력 채널( $k$ )마다 스케일  $s_{X,k}$ 가 다르다면 어떻게 될까요?

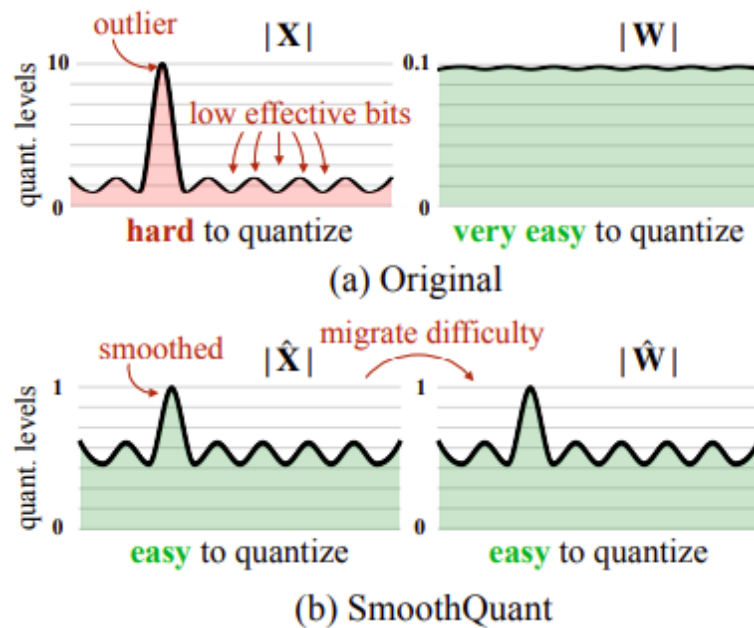
$$y_{tj} = \sum_{k=1}^{C_i} (x_{tk} \cdot s_{X,k} \cdot w_{kj})$$

이 경우  $s_{X,k}$ 는  $k$ 가 변함에 따라 계속 같이 변합니다. **시그마 밖으로 나갈 수가 없습니다.**

**결론:** 덧셈을 한 번 할 때마다 매번 스케일을 곱해야 하므로, "정수끼리 일단 다 더하고 나중에 변환한다"는 양자화의 대전제가 깨집니다.

즉, outer dimension인 weight의 channel에 대해서는 scale factor를 분리하여, 마지막에 한번에 적용할 수 있지만, inner dimension인 activation의 channel은 이것이 불가능하기 때문에, hardware에 unfriendly하다. (가능은 하지만, 매우 느려진다.)

[SmoothQuant]



이에 SmoothQuant는 activation quantization difficulty를 weight quantization과 분담하는 방법을 제시 (activation은 smoothing factor  $s$ 로 나누고, weight에는  $s$ 를 곱해서 output은 동일하지만, outlier 값을 좀 더 smooth하게 변환)

$$\mathbf{Y} = (\mathbf{X} \text{diag}(s)^{-1}) \cdot (\text{diag}(s) \mathbf{W}) = \hat{\mathbf{X}} \hat{\mathbf{W}}$$

위의 식과 같이 outlier가 큰 activation을 줄이기 위해  $s$ 로 나누고, 출력을 유지하기 위해 그만큼  $s$ 를 weight에 곱하여 activation의 quantization difficulty를 분담한다.

이를 Offline(Calibration set)에서 미리 계산해두어 extra scaling에 대해 overhead X

- **Smoothing Factor  $s$**

단순히  $s$ 를  $\max(X_j)$ , 즉 activation의 max 값으로 smoothing하는 것은 오히려 weight에 quantization에 대한 부담을 전부 주기 때문에, 이를 적절히 나눠 가져야 함.  
즉, quantization difficulty를 적절히 나눠서 모두 quantization이 easy하도록!

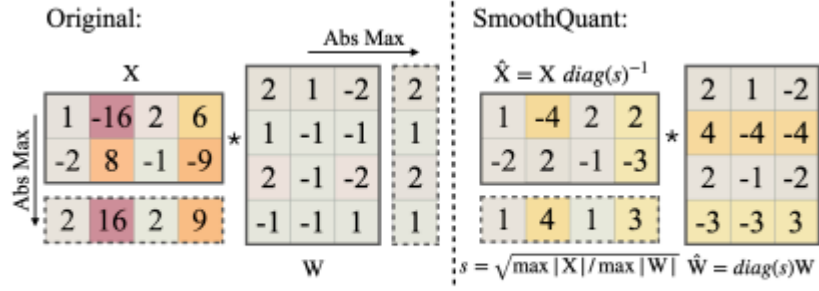


Figure 5: Main idea of SmoothQuant when  $\alpha$  is 0.5. The smoothing factor  $s$  is obtained on calibration samples and the entire transformation is performed offline. At runtime, the activations are smooth without scaling.

Here we introduce a hyper-parameter, migration strength  $\alpha$ , to control how much difficulty we want to migrate from activation to weights, using the following equation:

$$s_j = \max(|\mathbf{X}_j|)^\alpha / \max(|\mathbf{W}_j|)^{1-\alpha} \quad (4)$$